The Original Problem: Turn a User Command into Safer Behavior

Long-term research question

Can a user describe both a task and a safety requirement in natural language, and can that requirement be translated into STL and used during Safe RL training?

Example command

"Reach the goal. If you get too close to an obstacle, return to a safe distance within 10 steps."

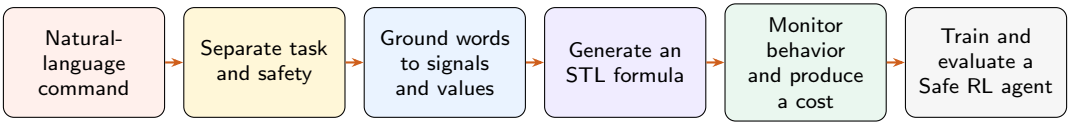
Task objective

Reach the goal efficiently.

Temporal safety requirement

After a warning, recover before a deadline.

The Complete Research Chain Has Six Links



In the example

Goal reaching remains the task reward. "Too close," "safe distance," and "10 steps" must become measurable conditions.

How STL reaches RL

The formula is not passed to RL as text. A monitor evaluates the trajectory and converts the result into a safety cost.

Each Link Introduces a Different Source of Uncertainty

1. Language interpretation

Did the system identify the intended task and safety requirement?

2. Grounding

Do words such as "close" refer to the correct signal, threshold, and time unit?

3. Monitoring and cost design

Does the STL monitor judge trajectories correctly, and does its output provide a useful training signal?

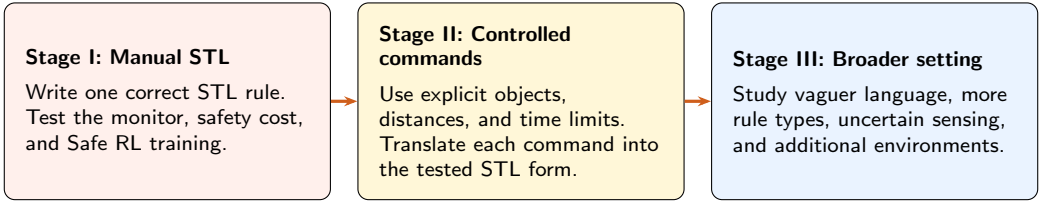
4. Safe RL learning

Can the agent reduce temporal safety failures without losing the ability to complete the task?

Why the full experiment is difficult to interpret

Any one component can fail even when the others are correct. An end-to-end failure would not reveal which research problem caused it.

We Therefore Split the Research into Three Stages



Current focus

Stage I isolates the downstream question: if the safety rule is already correct, can it be used effectively in Safe RL?

Stage I Uses One Concrete Navigation Benchmark

Application

A simulated mobile robot navigates through a known 2D workspace, reaches a target, and responds when it gets too close to a static hazard.

Selected benchmark

`SafetyPointGoal1-v0`

Why this is the first test

- It already supports goal-directed Safe RL.
- Hazard distance can be obtained from simulator state.
- Its native hazard cost gives a direct comparison.
- Static hazards keep the first experiment easy to interpret.

Deliberate trade-off

This benchmark is less realistic than UAV, driving, or real-robot settings, but it minimizes unrelated engineering and perception uncertainty in the first test.

Stage I Tests One Bounded-Recovery Rule

Rule in plain language

If the agent enters a warning zone, it must return to a safe distance within K steps.

One STL representation

$$\mathbf{G}\big(d_t < d_{\mathrm{warn}} \;\rightarrow\; \mathbf{F}_{[0,K]}(d_t \geq d_{\mathrm{safe}})\big)$$

Choose $d_{\mathrm{warn}} < d_{\mathrm{safe}}$; fix all values before training.

The experiment in four steps

1. Define the distance signal and feasible rule values.
2. Check the rule on clear success and failure paths.
3. Train agents in the same task with different safety methods.
4. Compare temporal failures and goal completion.

Controlled comparison: keep the environment and task fixed; change only the safety method.

The Comparison Separates Temporal Safety from Simpler Alternatives

Method	Advantage	Limitation or role
Task reward only	Minimal reference condition	No explicit safety objective
Native hazard cost	Built in and easy to use	Instantaneous; cannot express a recovery deadline
Temporal STL cost	Explicit deadline and reusable monitor	Needs measurable signals, parameters, and a correct monitor
Hand-coded timer	Direct implementation	Useful check, but each new rule needs custom logic

Decision to proceed

Move to Stage II only if the STL rule is measured correctly and reduces recovery failures without an unacceptable loss in goal completion.

A Successful Stage I Result Has a Narrow, Clear Meaning

What the experiment can address

- Simulated navigation around known static hazards.
- Rules based on a directly measurable numeric signal.
- Bounded responses: after an event, recover before a deadline.

What it cannot establish

- Translation of vague or incomplete language.
- Robustness to perception noise, moving hazards, or multiple agents.
- Real-world safety or a guarantee of zero violations.

Interpretation

Success would verify the downstream STL-to-Safe-RL path in one controlled setting. It would justify adding language, not claim that the complete language-based system is solved.

Stage I Creates a Stable Starting Point for Stage II

What Stage I should deliver

- One fixed navigation task and safety signal.
- One tested STL rule and trajectory monitor.
- Evidence about whether the temporal cost changes learned behavior.

What Stage II then adds

- Controlled commands with explicit distances and deadlines.
- A language-to-STL translation component.
- Separate evaluation of translation accuracy and downstream behavior.

Immediate next step

Freeze a one-page Stage I definition: the task, distance signal, STL rule, initial parameters, and one clear success and failure trajectory.